

```

import streamlit as st
import asyncio
import os
from openai import OpenAI
from yt_dlp import YoutubeDL
from moviepy.editor import VideoFileClip, AudioFileClip, vfx
import edge_tts

# Interface amigável
st.set_page_config(page_title="Shopee Video Generator", page_icon="❤️")
st.title("❤️ Gerador Automático de Vídeos Shopee")
st.info("Transforme qualquer vídeo bruto em um comercial pronto com narração IA.")

# Barra lateral para chaves
with st.sidebar:
    st.header("Configurações")
    api_key = st.text_input("Sua OpenAI API Key", type="password")
    voz = st.selectbox("Escolha a Voz", ["pt-BR-FranciscaNeural", "pt-BR-DonatoNeural"])

# Inputs do usuário
url = st.text_input("Link do vídeo (TikTok, YouTube, etc)")
produto = st.text_input("Nome do produto")

if st.button("🚀 GERAR VÍDEO AGORA"):
    if not api_key or not url or not produto:
        st.warning("Preencha todos os campos antes de continuar.")
    else:
        try:
            client = OpenAI(api_key=api_key)

            with st.status("🎬 Processando seu vídeo...", expanded=True) as status:
                # 1. Roteiro
                st.write("🧠 Criando roteiro de vendas...")
                prompt = f'Roteiro de 15 seg para {produto}. Foque em benefício e urgência. Termine com \'Link na bio\''
                res = client.chat.completions.create(model="gpt-3.5-turbo", messages=[{"role": "user", "content": prompt}])
                roteiro = res.choices[0].message.content

                # 2. Voz
                st.write("🗣️ Gravando locução...")
                audio_temp = "temp_voz.mp3"
                async def make_voice():
                    c = edge_tts.Communicate(roteiro, voz)
                    await c.save(audio_temp)
                asyncio.run(make_voice())

                # 3. Download

```

```

st.write("🍰 Baixando vídeo...")
video_temp = "temp_video.mp4"
with YoutubeDL({'outtmpl': video_temp, 'format': 'best[ext=mp4]'}) as ydl:
    ydl.download([url])

# 4. Edição
st.write("✂️ Editando e sincronizando...")
clip = VideoFileClip(video_temp).without_audio().fx(vfx.mirror_x)
audio_clip = AudioFileClip(audio_temp)

# Sincroniza e corta
final_name = f"{produto.replace(' ', '_')}_final.mp4"
clip = clip.set_duration(audio_clip.duration + 0.5).set_audio(audio_clip)

# Ajuste de Formato Vertical
w, h = clip.size
if w/h > 0.6:
    clip = clip.crop(x_center=w/2, width=h*0.5625, height=h)

clip.write_videofile(final_name, codec="libx264", audio_codec="aac", fps=24,
logger=None)
status.update(label="✅ Vídeo Criado com Sucesso!", state="complete")

# Resultado
st.video(final_name)
with open(final_name, "rb") as f:
    st.download_button("🍰 BAIXAR VÍDEO", f, file_name=final_name)

except Exception as e:
    st.error(f"Erro: {e}")

```